

Toy Inference From a Saved Run Directory

Mie Spray Postprocessing Project

2026 年 3 月 20 日

Audience: Engineers who want a quick post-training sanity check without rebuilding the full dataset.

Goal of this talk:

- Explain what `toy_inference_from_run.ipynb` does.
- Show how manual physical inputs are transformed into model-compatible features.
- Interpret the notebook outputs: mean penetration, uncertainty band, and simple collision proxies.
- Clarify what is useful now and what is still only a geometric approximation.

1.1 Notebook Workflow

Core workflow:

- ① Set RUN_DIR and manual physical inputs.
- ② Load train_config_used.json, scaler_state.json, and model checkpoint.
- ③ Rebuild the exact training-time feature scaling.
- ④ Sweep time from 0 ms to 5 ms and predict $\mu(t)$ and $\log \sigma^2(t)$.
- ⑤ Visualize uncertainty and project the result into a simple 2D chamber geometry.

1.2 Run Artifacts Used by the Notebook

Observed default run directory in the notebook:

- MLP/runs_mlp/stage2_NLL_penetration_20260317_194155

Artifacts loaded from that run:

- network architecture and hyperparameters from `train_config_used.json`
- scaling statistics from `scaler_state.json`
- model weights from the best available checkpoint

Observed notebook output:

- device: **cuda**
- feature dimension: **9**
- checkpoint found in the example run: `best_model_stage1.pt`

2.1 Example Toy Input Condition

Default manual inputs in the notebook:

- tilt angle = 20°
- plumes = 10
- nozzle diameter = 0.355 mm
- injection duration = 1000 μ s
- injection pressure = 2000 bar
- chamber pressure = 25 bar
- control backpressure = 4 bar

Time sweep:

- 300 points over 0 ms to 5 ms

2.2 How Raw Inputs Are Scaled

The notebook reuses the same preprocessing logic as training:

- time is normalized to $[0, 1]$ using the saved min/max
- pressure terms use $\log P_{\text{inj}}$, $\log P_{\text{ch}}$, and $\log(P_{\text{inj}} - P_{\text{ch}})$
- geometry and condition terms are z-scored using the saved train statistics

Resulting feature vector per time point:

$$x_t = [\text{time_norm}, \text{tilt}_z, \text{plumes}_z, \text{diameter}_z, \text{duration}_z, \log P_{\text{inj},z}, \log P_{\text{ch},z}, \log \Delta P_z, \text{backpressure}_z]$$

This guarantees inference uses the same input space as training.

3.1 What the MLP Predicts

The loaded model outputs two channels:

$$[\mu(t), \log \sigma^2(t)]$$

Post-processing in the notebook:

$$\sigma(t) = \max\left(\sqrt{e^{\log \sigma^2(t)}}, \sigma_{\min}\right)$$

Then the notebook plots:

- predicted mean penetration $\mu(t)$
- predictive interval $\mu(t) \pm \sigma(t)$
- an optional 90 mm reference line
- where $\sigma_{\min}$ is the saved floor from `std_clamp_min`

3.2 Example Numerical Output

Observed time-sweep output from the notebook:

- feature matrix shape: **(300, 9)**
- predicted std range: **6.27 mm to 15.15 mm**

Example single-time query from the same run at 0.5 ms:

- predicted mean penetration $\mu \approx 32.74$ mm
- predicted axial std $\sigma \approx 10.18$ mm

This is the simplest way to check whether the Stage-2 model produces plausible uncertainty magnitudes under manually chosen conditions.

4.1 Simple Mask-Based Overlap Solver

After 1D inference, the notebook builds a coarse 2D geometry:

- cylinder radius = 120 mm
- cylinder height = 60 mm
- cylinder head offset = 50 mm
- grid size = 1 mm
- wall mask starts at $x \geq 120$ mm

A simple deterministic proxy is formed by:

- projecting the predicted penetration along the spray tilt direction
- turning the predicted mean length and $+1\sigma$ length into binary masks
- measuring overlap area with wall and cylinder masks

4.2 Gaussian Collision Proxy

The notebook then upgrades the simple mask idea to a rotated 2D Gaussian:

$$p(u, v) \propto \exp\left(-\frac{1}{2} \left[\left(\frac{u}{\sigma_{\text{axis}}}\right)^2 + \left(\frac{v}{\sigma_{\perp}}\right)^2 \right]\right)$$

How the widths are chosen:

- axial spread: $\sigma_{\text{axis}} = \sigma$ from model output
- orthogonal spread: plume width $\approx \mu \tan(\theta/2)$
- notebook assumption: $\sigma_{\perp} \approx \text{width}/3$

The Gaussian is rotated by the spray tilt angle and integrated over wall/cylinder masks to get probability mass.

4.3 Example Geometry Result

Observed notebook outputs for the default test condition:

- simple wall collision area: **0**
- simple cylinder collision area: **0**
- $+1\sigma$ simple overlaps: also **0**

Gaussian-mask probability masses:

- wall mass $\approx 1.07 \times 10^{-20}$
- cylinder mass $\approx 6.09 \times 10^{-23}$
- any collision mass $\approx 1.08 \times 10^{-20}$

For this operating condition, the toy case is effectively far from collision.

4.4 What This Notebook Is Good For

Current strengths:

- fast sanity check for trained runs
- manual sensitivity study over pressure, duration, tilt, and diameter
- quick interpretation of predicted uncertainty in physical units
- simple geometry screening before building a more complete solver

Current limitations:

- 2D spread is assumed, not learned directly from images
- orthogonal Gaussian width is a heuristic from cone angle
- collision metric is only a proxy, not a CFD or multiphase transport model

Takeaway

- `toy_inference_from_run.ipynb` turns a saved run directory into a practical engineering demo tool.
- It exactly reuses the saved scaler and model interface, so toy inputs stay training-compatible.
- The notebook does more than draw $\mu \pm \sigma$: it also maps 1D uncertainty into a simple 2D collision proxy.
- This makes it a useful bridge between learned penetration prediction and geometry-aware downstream reasoning.